Image Reconstruction with One-Step Drifting Models

Aviram Ohayon (ID 315997866) **Ziv Moshe** (ID 315527176)

Advanced Topics in Signal and Image Processing — Final Project

Abstract

Image-Adaptive GAN (IAGAN) lowers the representation error of a generative prior by fine-tuning the generator's weights on each test measurement. In this project we port this recipe to a pretrained one-step pixel-space drifting model. We first invert the generator by optimizing its image-sized Gaussian input (CSGM), and then free its weights as well (IADrift). We test on noiseless $\times 4$ super-resolution of 100 held-out ImageNet-V2 images, using back-projection and comparisons fixed before testing. Early-stopped adaptation gains only 0.008 LPIPS and loses 0.12 dB PSNR against CSGM. Without early stopping it loses 2.92 dB, on every image. Notice that fitting the measurements better does not give a better image: during adaptation the low-resolution loss falls more than tenfold while PSNR drops. The frozen generator has a large representation error, and adapted weights can remove it. However, the measurements do not constrain the null space of the downsampling operator, so the weight updates change that content with no information about the true image. What does help is averaging. A fit-weighted mean of five CSGM reconstructions, an approximate posterior mean fixed before testing, gains 1.57 dB and 0.032 LPIPS over CSGM. The uniform mean (secondary) gains 2.13 dB; it stays 0.69 dB below bicubic in PSNR but is much better in LPIPS (0.247 against 0.358).

1 Introduction

Let us consider the linear inverse problem $\mathbf{y} = \mathbf{A}\mathbf{x}^\star + \mathbf{e}$. When $\mathbf{A}$ has fewer rows than columns, many images explain $\mathbf{y}$ equally well, therefore a prior is needed to choose among them. CSGM (Bora et al., 2017) searches the latent space of a pretrained generator for the image that fits $\mathbf{y}$ best. Its main weakness is the generator's *representation error*, the distance from the true image to the closest image the generator can produce. Image-Adaptive GAN (IAGAN) (Abu Hussein et al., 2020) reduces it by fine-tuning the generator's weights on each test measurement, starting from the CSGM solution.

Diffusion and flow priors generate by an iterative chain of network evaluations, which a solver must run or differentiate through. Drifting models (Deng et al., 2026) are one-step generators, so they could bring back the cheap test-time optimization of CSGM and IAGAN with a modern prior (a motivation only; we ran no diffusion-based solver). Concurrent work, DriftSR (Zhu et al., 2026), trains a supervised super-resolution network with drifting; we instead invert a pretrained drifting generator per image. Unlike a GAN's low-dimensional latent ($k \ll n$), the pixel-space drifting model's input is image-sized noise plus 32 discrete style labels that gradients cannot optimize, so it is not clear in advance whether the IAGAN recipe carries over. On noiseless $\times 4$ super-resolution, our contributions are[1]:

1. IADrift, evaluated with a protocol fixed before testing (Sections 3, 4.2): early-stopped adaptation trades a little PSNR for a little LPIPS.
2. A diagnosis of why (Section 4.3): CSGM's gap is largely representation error, but under SR the freedom of the weights goes into the null space of $\mathbf{A}$.
3. A posterior-mean ensemble of CSGM reconstructions that improves PSNR and LPIPS over CSGM and LPIPS over bicubic, and a test of the gain $2/(1 + 1/K)$ predicted for averaging K posterior samples (Section 4.4).

Deviations from the proposal. We used ImageNet-V2 instead of the ImageNet-1k validation set, and did not run SR$\times$8 or noisy SR: on SR$\times$4, the easiest planned setting, CSGM stayed well below bicubic and adaptation lowered PSNR further, so we first investigated why.

[1] A full report, with everything we tried, is available at `https://github.com/zivmoshe12/IADrift/blob/HEAD/docs/extended_report.pdf`.

2 The Existing Method

We observe $\mathbf{x}^\star \in \mathbb{R}^n$ as $\mathbf{y} = \mathbf{A}\mathbf{x}^\star + \mathbf{e}$ with $\mathbf{A} \in \mathbb{R}^{m\times n}$, $m < n$, and $\mathbf{e} = \mathbf{0}$ (noiseless). For super-resolution $\times 4$ (SR$\times$4), $\mathbf{A}$ is antialiased bicubic downsampling from $256 \times 256 \times 3$ to $64 \times 64 \times 3$, so $n = 196{,}608$, $m = 12{,}288$ and the null space of $\mathbf{A}$ has a dimension of at least $n - m = 184{,}320$. $\mathbf{y}$ mainly constrains the content below the low-resolution Nyquist limit $f_{\mathrm{LR}} = 1/8$ cycles/pixel (softly, since the bicubic kernel is not an ideal low-pass filter), so content above f_{LR} must come mostly from a prior.

2.1 CSGM and image-adaptive GANs

Given a pretrained generator $G_{\boldsymbol{\theta}_0} : \mathbb{R}^k \to \mathbb{R}^n$ with $k \ll n$, *CSGM* solves the first problem below by gradient descent through G; its accuracy is limited by the representation error $E_{\mathrm{rep}}(G, \mathbf{x}) = \min_{\mathbf{z}} \|G(\mathbf{z}) - \mathbf{x}\|_2$. *IAGAN* (Abu Hussein et al., 2020) then also frees the weights, starting from $(\boldsymbol{\theta}_0, \hat{\mathbf{z}})$:

$$\hat{\mathbf{z}} = \arg\min_{\mathbf{z}\in\mathbb{R}^k} \|\mathbf{y} - \mathbf{A}G_{\boldsymbol{\theta}_0}(\mathbf{z})\|_2^2, \qquad (\hat{\boldsymbol{\theta}}, \hat{\mathbf{z}}_{\mathrm{IA}}) = \arg\min_{\boldsymbol{\theta},\mathbf{z}} \|\mathbf{y} - \mathbf{A}G_{\boldsymbol{\theta}}(\mathbf{z})\|_2^2, \tag{1}$$

with outputs $G_{\boldsymbol{\theta}_0}(\hat{\mathbf{z}})$ and $G_{\hat{\boldsymbol{\theta}}}(\hat{\mathbf{z}}_{\mathrm{IA}})$, a small learning rate and early stopping, so that the semantic prior is kept while the specific image becomes representable. For a random Gaussian $\mathbf{A}$ the CSGM error is bounded by $6E_{\mathrm{rep}}$ plus noise and optimization terms, and freeing the first layer can only lower E_{rep} (Abu Hussein et al., 2020, Theorem 1, Prop. 2); neither result covers our bicubic $\mathbf{A}$, so we use them only as motivation. For low noise they also enforce hard consistency with $\mathbf{y}$ by back-projection (BP) onto $\{\mathbf{x} : \mathbf{A}\mathbf{x} = \mathbf{y}\}$, which we solve with conjugate gradients:

$$\mathrm{BP}(\hat{\mathbf{x}}) = \arg\min_{\mathbf{x}} \|\mathbf{x} - \hat{\mathbf{x}}\|_2 \ \text{ s.t. } \mathbf{A}\mathbf{x} = \mathbf{y} \ = \ \hat{\mathbf{x}} + \mathbf{A}^\dagger(\mathbf{y} - \mathbf{A}\hat{\mathbf{x}}), \qquad \mathbf{A}^\dagger = \mathbf{A}^\top(\mathbf{A}\mathbf{A}^\top)^{-1}. \tag{2}$$

Let us denote $P_{\mathbf{A}} = \mathbf{A}^\dagger\mathbf{A}$ and $Q_{\mathbf{A}} = \mathbf{I} - P_{\mathbf{A}}$. Then $\mathrm{BP}(\hat{\mathbf{x}}) = P_{\mathbf{A}}\mathbf{x}^\star + Q_{\mathbf{A}}\hat{\mathbf{x}} + \mathbf{A}^\dagger\mathbf{e}$, meaning BP removes the error in the row space of $\mathbf{A}$ and leaves the null-space component of $\hat{\mathbf{x}}$ untouched.

2.2 Drifting models

Deng et al. (2026) train a one-step generator $f_{\boldsymbol{\theta}}$ so that its pushforward $q = f_{\boldsymbol{\theta}\#}p_{\boldsymbol{\varepsilon}}$ matches p_{data}, using the fixed-point loss

$$\mathcal{L}_{\mathrm{drift}} = \mathbb{E}_{\boldsymbol{\varepsilon}} \sum_j \Big\| \phi_j(\mathbf{x}) - \mathrm{sg}\big(\phi_j(\mathbf{x}) + \mathbf{V}_{p,q}(\phi_j(\mathbf{x}))\big) \Big\|_2^2, \qquad \mathbf{x} = f_{\boldsymbol{\theta}}(\boldsymbol{\varepsilon}), \tag{3}$$

where ϕ_j are encoder features, sg is the stop-gradient, and the drifting field $\mathbf{V}_{p,q} = \mathbf{V}_p^+ - \mathbf{V}_q^-$ is the difference of kernel mean-shift vectors towards data and generated samples, which vanishes when $q = p$ (Deng et al., 2026, Eqs. 8, 10, 14). We use the released pixel-space model `pixel_B_sota`, a DiT-style B/16 Transformer with 134M parameters (Deng et al., 2026, Table 6, App. A.2), $\mathbf{x} = G_{\boldsymbol{\theta}}(\boldsymbol{\varepsilon}, \mathbf{s};\, c)$ with $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \in \mathbb{R}^{256\times256\times3}$ cut into 256 patch tokens of 16×16, 32 discrete *style labels*[2] $\mathbf{s} \in \{0, \dots, 63\}^{32}$ that index a learned codebook, and the class $c \in \{0, \dots, 999\}$. We fix the guidance scale $\alpha = 1$ (no guidance) and use the true class.

2.3 Advantages and limitations

IAGAN lowers the representation error without retraining, keeps the semantic prior by starting from the CSGM solution, works for any differentiable $\mathbf{A}$ with BP, and costs about 17% extra run time in their compressed-sensing example (Abu Hussein et al., 2020); but the measurements cannot choose its learning rate or stopping step (Section 4.3), it can overfit the null space when m is small, and it needs a copy of the weights per image. *Drifting* needs one forward and one backward pass per

[2]Called `noise_labels` in our code and *random style embeddings* by Deng et al. (2026).

CSGM or IAGAN step instead of a chain of evaluations, has no adversarial game, and reaches FID 1.76 on ImageNet 256×256 (Deng et al., 2026, Table 6), with a searched, unreported guidance scale, so possibly not for our $\alpha = 1$; but it has no low-dimensional latent, no guarantee for an optimized $\boldsymbol{\varepsilon}$ off the Gaussian shell, discrete labels that gradients cannot reach, and a batch-level objective (3) that per-image inversion does not use.

3 Extensions

The proposal named two stages: CSGM on the drifting prior, then IAGAN's weight adaptation. A third, averaging several stage-1 solutions as an approximate posterior mean, came out of development. All of them minimize the same *low-resolution loss*, and differ in which variables are free:

$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\varepsilon}; \mathbf{s}) \;=\; \big\|\mathbf{y} - \mathbf{A}\, G_{\boldsymbol{\theta}}(\boldsymbol{\varepsilon}, \mathbf{s};\, c)\big\|_2^2. \tag{4}$$

(The shifted ensemble members of Section 3.3 minimize it for a shifted $\mathbf{y}$.) No step uses $\mathbf{x}^\star$: the high-resolution (HR) image is used only for scoring.

3.1 Stage 1: CSGM on the Gaussian input

The weights are frozen at $\boldsymbol{\theta}_0$ and the labels are discrete, therefore CSGM optimizes only $\boldsymbol{\varepsilon}$. For restarts $r \in \{0, 1\}$ we draw $\boldsymbol{\varepsilon}_r^{(0)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and $\mathbf{s}_r \sim \mathrm{Unif}(\{0, \dots, 63\}^{32})$, and keep the restart that fits $\mathbf{y}$ best:

$$\hat{\boldsymbol{\varepsilon}}_r = \arg\min_{\boldsymbol{\varepsilon}} \mathcal{L}(\boldsymbol{\theta}_0, \boldsymbol{\varepsilon}; \mathbf{s}_r), \quad r^\star = \arg\min_r \mathcal{L}(\boldsymbol{\theta}_0, \hat{\boldsymbol{\varepsilon}}_r; \mathbf{s}_r), \quad \hat{\mathbf{x}}_{\mathrm{CSGM}} = G_{\boldsymbol{\theta}_0}(\hat{\boldsymbol{\varepsilon}}_{r^\star}, \mathbf{s}_{r^\star}; c). \tag{5}$$

Each argmin is approximated by 5,000 Adam steps with learning rate 0.02 and no input regularizer, a setting chosen by a grid search on 100 development images (learning rate $\in \{0.005, 0.02\} \times$ no regularizer or a moment penalty on $\boldsymbol{\varepsilon}$ with weight $\in \{1, 10, 100\}$; 3,000 steps, ranked by the low-resolution loss; the step count was then raised to 5,000 before testing).

3.2 Stage 2: IADrift, image-adaptive drifting generator

Following (1), we free the weights as well, starting from the CSGM solution and keeping its style labels $\mathbf{s}^\star := \mathbf{s}_{r^\star}$:

$$(\hat{\boldsymbol{\theta}}, \hat{\boldsymbol{\varepsilon}}) \in \arg\min_{\boldsymbol{\theta}, \boldsymbol{\varepsilon}} \; \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\varepsilon}; \mathbf{s}^\star), \qquad (\boldsymbol{\theta}^{(0)}, \boldsymbol{\varepsilon}^{(0)}) = (\boldsymbol{\theta}_0, \hat{\boldsymbol{\varepsilon}}_{r^\star}). \tag{6}$$

We use two Adam optimizers, with learning rates $\eta_\theta = 10^{-5}$ on all 134M weights and $\eta_\varepsilon = 10^{-3}$ on the input. We write IADrift@T for the output after T IADrift steps (joint Adam steps on $\boldsymbol{\theta}$ and $\boldsymbol{\varepsilon}$) from the CSGM solution, and report two outputs of one 1,000-step run: **IADrift@100**, after 100 steps (early-stopped), and **IADrift@1000**, after 1,000 steps (without early stopping). On the development images, joint IADrift raised PSNR early and then lowered it (recorded before testing). Our exported traces adapt only $\boldsymbol{\theta}$, which ends close to the joint update (Section 4.3): PSNR without BP rose by $+0.28$ dB at step 50 (the peak on three of four images, step 100 on the fourth) and then fell. Choosing the peak needs the HR image, so we declared the round step 100 before testing. A smaller learning rate $\eta_\theta = 10^{-6}$ gave a smaller peak ($+0.05$ dB) and only slowed the decline (-1.2 against -2.7 dB at step 1,000).

3.3 Posterior-mean ensemble

$\mathbf{y}$ does not constrain the null space of $\mathbf{A}$, so CSGM runs from different random starts give different null-space content, and averaging them approximates the posterior mean. If the truth and the reconstructions were independent posterior samples, $\mathbf{x}^\star = \boldsymbol{\mu} + \mathbf{u}^\star$ and $\mathbf{x}_k = \boldsymbol{\mu} + \mathbf{u}_k$ with $\boldsymbol{\mu} = \mathbb{E}[\mathbf{x} \mid \mathbf{y}]$ and $\mathbf{u}^\star, \mathbf{u}_1, \dots, \mathbf{u}_K$ i.i.d. with zero mean and covariance Σ, then

$$\mathbb{E}\|\mathbf{x}_k - \mathbf{x}^\star\|^2 = 2\,\mathrm{tr}\,\Sigma, \qquad \mathbb{E}\Big\|\tfrac{1}{K}\textstyle\sum_{k=1}^K \mathbf{x}_k - \mathbf{x}^\star\Big\|^2 = \big(1 + \tfrac{1}{K}\big)\mathrm{tr}\,\Sigma, \tag{7}$$

a gain of $10\log_{10}\frac{2}{1+1/K}$, i.e. 2.22 dB for $K=5$. With a pairwise correlation $\mathbb{E}[\mathbf{u}_k^\top \mathbf{u}_l] = \rho \operatorname{tr}\Sigma$ ($k \neq l$), the ensemble error becomes $\big(1 + \frac{1}{K} + \frac{(K-1)\rho}{K}\big)\operatorname{tr}\Sigma$. CSGM members are fits to $\mathbf{y}$, not posterior samples, so this model only motivates the averaging.

Our $K=5$ members are the two CSGM restarts and three CSGM runs on circularly shifted measurements, each started from restart 0's $(\boldsymbol{\varepsilon}_0^{(0)}, \mathbf{s}_0)$:

$$\mathbf{x}_{\mathbf{d}} = S_{-4\mathbf{d}}\,\mathrm{CSGM}\big(S_{\mathbf{d}}\mathbf{y}\big), \qquad \mathbf{d} \in \{(0,1),(1,0),(1,1)\}, \tag{8}$$

where $S_{\mathbf{d}}$ shifts by $\mathbf{d}$ pixels. A shift by 4 HR pixels commutes with $\mathbf{A}$ up to the border, so each shifted run is again a fit to $\mathbf{y}$; the generator is not shift-equivariant, so from the same start it reaches a different fit. Weighting the members by their measurement fit gives ENS-W:

$$\hat{\mathbf{x}}_{\text{ENS-W}} = \mathrm{BP}\Big(\sum_{k=1}^{5} w_k \mathbf{x}_k\Big), \qquad w_k = \frac{\exp\big(-(L_k - L_{\min})/\tau\big)}{\sum_j \exp\big(-(L_j - L_{\min})/\tau\big)}, \tag{9}$$

where $L_k = \|\mathbf{y} - \mathbf{A}\mathbf{x}_k\|_2^2$ is member k's low-resolution loss and $\tau = \mathrm{std}(L_1, \ldots, L_5)$; equal weights give ENS-U. We also report the average of the two restarts, which costs nothing beyond CSGM. To test (7), we ran a late-stage exploratory experiment on 6 development images, designed after the test runs (Appendix A). Each image has 12 CSGM members, each from its own $(\boldsymbol{\varepsilon}_r^{(0)}, \mathbf{s}_r)$, saved after $T \in \{500, \ldots, 5{,}000\}$ steps; every 5,000-step member is also continued with IADrift@100. We average up to 20 random K-subsets for $K \leq 12$ and compare the gain $10\log_{10}(\overline{\mathrm{MSE}}_{\mathrm{mem}}/\mathrm{MSE}_{\mathrm{ens}})$ before BP, per image, with the prediction.

4 Results

4.1 Experimental setup

Data. The generator was trained on ImageNet, so we use ImageNet-V2 MatchedFrequency, collected independently with the same 1,000 classes, split per class into 8 development and 2 test images. The test set is 100 images drawn at random, with a fixed seed, from 100 distinct classes of the test split, centre-cropped to 256×256. All settings were chosen on development images (100 for the CSGM sweep, 4 for the diagnostics, 6 for the ensembles); one test image was used only to check that the code runs.

Metrics. PSNR (data range 2) and LPIPS (AlexNet), averaged over images, and the band SNR $10\log_{10}(\|P_b\mathbf{x}^\star\|^2/\|P_b(\hat{\mathbf{x}} - \mathbf{x}^\star)\|^2)$, where P_b keeps the radial frequencies of band b (of the Hann-windowed spectrum): low $[0, \frac{1}{16})$ and mid $[\frac{1}{16}, \frac{1}{8})$ below f_{LR}, high $[\frac{1}{8}, \frac{1}{4})$ and very high $[\frac{1}{4}, \infty)$ above it. An empty band scores 0 dB, meaning a negative value indicates that the invented content is worse than none.

Protocol. Before IADrift ran on any test image we fixed four primary tests: IADrift@100 and IADrift@1000 against CSGM, after BP, in PSNR and LPIPS, each paired over the 100 images with a 95% CI (mean $\pm 1.96\,\hat{\sigma}/\sqrt{100}$ of the per-image differences), a Wilcoxon p Holm-corrected over the four tests, and the number of images on which the method is better. The ensemble was added afterwards, but ENS-W against CSGM, in PSNR and LPIPS, was fixed the same way before any ensemble ran on test images (Holm over these two tests). All other comparisons, including ENS-U, are reported with the same statistics but are not corrected.

4.2 Main result: early-stopped adaptation trades PSNR for LPIPS

The existing method on this prior. CSGM + BP is 2.82 dB below bicubic + BP, on all 100 images, but better in LPIPS (on 76/100; Table 1): the generator adds plausible detail that is not the true detail, the perception–distortion trade-off which Abu Hussein et al. (2020) also observe. BP raises PSNR on every image, as it must: it projects orthogonally onto an affine set that contains $\mathbf{x}^\star$.

Table 1: Test set, 100 images, all after BP. Δ: mean paired difference from CSGM with its 95% CI; # better: images on which the method beats CSGM. †: the four tests fixed before testing, all significant after Holm correction ($p \leq 1.4 \cdot 10^{-3}$); ‡: the two tests fixed before the ensemble test run ($p < 10^{-9}$). Steps: Adam steps through the generator per image (10k = two 5,000-step restarts); an IADrift step also updates the 134M weights and costs several CSGM steps. Bold: best.

				PSNR vs CSGM (dB)			LPIPS vs CSGM		
Method	Steps	PSNR ↑	LPIPS ↓	Δ	95% CI	# better	Δ	95% CI	# better
Bicubic	–	**25.57**	0.358	$+2.82$	$[+2.48, +3.16]$	100	$+0.074$	$[+0.049, +0.098]$	24
CSGM	10k	22.75	0.284	–	–	–	–	–	–
IADrift@100†	10.1k	22.63	0.276	-0.12	$[-0.20, -0.04]$	33	-0.008	$[-0.013, -0.004]$	72
IADrift@1000†	11k	19.83	0.366	-2.92	$[-3.18, -2.66]$	0	$+0.082$	$[+0.066, +0.097]$	12
Average of 2 restarts	10k	23.76	0.259	$+1.01$	$[+0.84, +1.19]$	89	-0.025	$[-0.034, -0.015]$	67
ENS-W‡	25k	24.32	0.252	$+1.57$	$[+1.40, +1.74]$	100	-0.032	$[-0.040, -0.024]$	76
ENS-U	25k	24.88	**0.247**	$+2.13$	$[+1.89, +2.37]$	100	-0.037	$[-0.051, -0.023]$	65

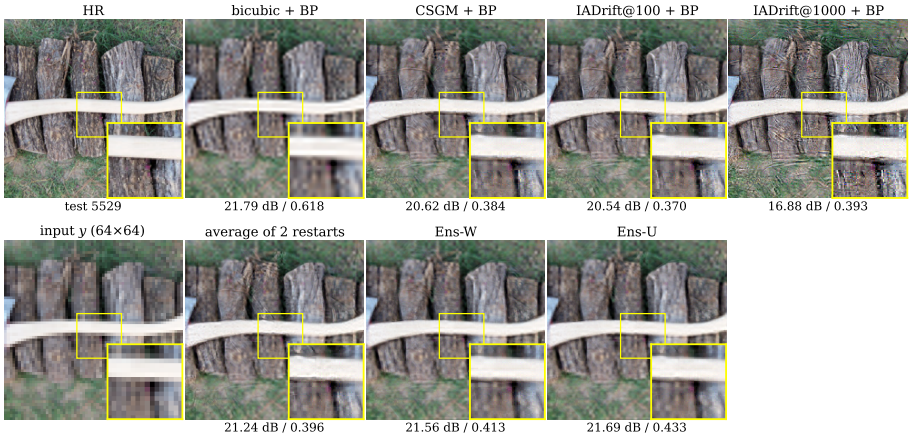


Figure 1: The second image of the fixed test sample (not chosen by result; the first was used to test the code), with every method of Table 1, all after BP. Second row: the measurement $\mathbf{y}$ (enlarged) and the three averages of CSGM runs. Below each image: PSNR / LPIPS. Inset: the yellow 64×64 crop.

IADrift. Without early stopping, adaptation clearly hurts: after 1,000 IADrift steps, PSNR is 2.92 dB lower than CSGM's, on every image (IADrift@1000). After 100 steps (IADrift@100), PSNR drops by 0.12 dB and LPIPS by 0.008, both significant but small. Without BP (secondary), IADrift@100 beats CSGM by $+0.28$ dB (81/100), mostly by a better fit of $\mathbf{y}$ in the row space, which BP gives every method for free. In Figure 1, CSGM adds sharp but wrong texture, IADrift@1000 adds more of it, and the averages remove much of it at the cost of some blur; on this image they have a worse LPIPS than CSGM (0.396–0.433 against 0.384), although on average they are better.

4.3 Why adaptation does not improve PSNR

Except for the band SNR and the low-resolution losses, these diagnostics use four development images and no tests (Table 2). Fitting $\mathbf{y}$ better does not give a better $\mathbf{x}$: during adaptation on the test set the median low-resolution loss falls from 38.5 to 0.98 (more than tenfold on 99/100 images), while PSNR after BP falls from 22.75 to 19.83 dB.

Representation or ill-posedness? We fit the HR image itself, replacing $(\mathbf{A}, \mathbf{y})$ by $(\mathbf{I}, \mathbf{x}^\star)$ in (4): not a reconstruction, only a measure of representation. The frozen generator reaches 24.4 dB, below bicubic + BP from $\mathbf{y}$ (25.4 dB); with adapted weights it reaches 40.1 dB, so adaptation *can* remove

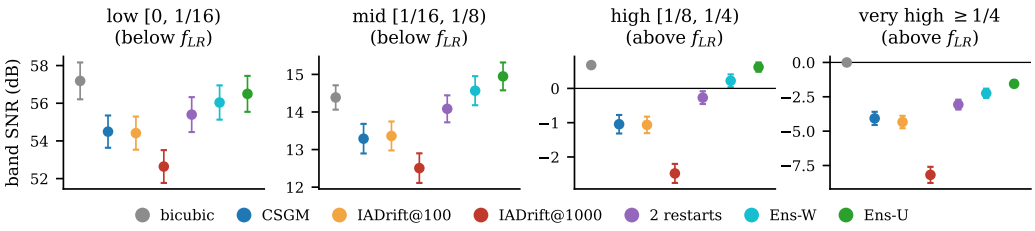


Figure 2: Band SNR on the test set (100 images, after BP; mean and 95% CI over images) for every method of Table 1, in the four bands of Section 4.1. 0 dB is what an empty band scores.

Table 2: Development diagnostics on 4 images, no tests; compare within a block only. HR fit ($\mathbf{A} = \mathbf{I}$): a measure of representation, not a reconstruction. Generated targets: $G_{\boldsymbol{\theta}_0}(\boldsymbol{\varepsilon}^\star, \mathbf{s}^\star; c)$, each scored against its own target. $\boldsymbol{\theta}$ only: IADrift with $\boldsymbol{\varepsilon}$ frozen.

Method	PSNR ↑	LPIPS ↓
4 images, SR		
Bicubic + BP	**25.43**	0.429
CSGM (no BP)	21.06	0.406
CSGM + BP	21.86	0.328
labels searched	22.10	**0.314**
$\boldsymbol{\varepsilon}$ projected per patch	21.99	0.373
$\boldsymbol{\varepsilon}$ projected on sphere	21.72	0.388
IADrift@1000 (no BP)	18.30	0.470
$\boldsymbol{\theta}$ only, step 1,000 (no BP)	18.36	–

Method	PSNR ↑	LPIPS ↓
Same 4 images, HR fit ($\mathbf{A} = \mathbf{I}$)		
CSGM-HR, frozen weights	24.44	0.332
IADrift-HR, adapted weights	**40.09**	**0.030**
4 generated targets (classes of the 4 images), SR		
Bicubic + BP	23.03	0.402
CSGM, true labels (no BP)	**25.93**	**0.116**
CSGM, random labels (no BP)	21.89	0.331

the representation error. On generated targets $\mathbf{x}^\star = G_{\boldsymbol{\theta}_0}(\boldsymbol{\varepsilon}^\star, \mathbf{s}^\star; c)$ with a random draw $(\boldsymbol{\varepsilon}^\star, \mathbf{s}^\star)$, which the generator can represent exactly, CSGM with the true labels reaches 25.9 dB without BP, above bicubic + BP (23.0 dB), but with random labels only 21.9 dB, although $\boldsymbol{\varepsilon}$ is free in both: on in-range targets, the random labels alone cost 4 dB. CSGM on the real photos is at a similar level (21.1 dB without BP), but we could not show the same effect there: choosing the labels by the low-resolution loss among 8 random sets per restart gains only 0.24 dB after BP (22.10 against 21.86 dB).

Where the freedom goes. After BP the error is $Q_{\mathbf{A}}(\hat{\mathbf{x}} - \mathbf{x}^\star)$ by (2), so for every method it lies in the null space; the methods differ in how much null-space error they add. The loss gradient with respect to the image, $-2\mathbf{A}^\top(\mathbf{y} - \mathbf{A}\hat{\mathbf{x}})$, lies in the row space, so for CSGM and IADrift alike the loss carries no information about $Q_{\mathbf{A}}\hat{\mathbf{x}}$, which is set by the generator. With frozen weights this content comes from the learned prior; adapting the weights moves the generator away from that prior as a side effect of fitting the row space, and nothing in the loss ties the move to $\mathbf{x}^\star$. The band SNR agrees (Figure 2): in the high / very high bands, bicubic + BP is near 0 dB (+0.7 / 0.0), CSGM + BP is below that (−1.0 / −4.1), and IADrift@1000 lower still (−2.5 / −8.2 dB). IADrift@1000 also loses SNR below f_{LR}, but all methods stay above 12 dB there. On the four test images saved as examples, 99.5% of the error energy that IADrift@1000 adds lies above f_{LR}, and 81% in the very-high band. The weights cause the drop: adapting only $\boldsymbol{\theta}$ gives 18.36 dB at step 1,000, against 18.30 dB for joint IADrift (both without BP), in line with IAGAN's caution about small m; here $m/n = 1/16$.

The proposal's technical core. The proposal expected the optimized $\boldsymbol{\varepsilon}$ to leave the Gaussian typical set. It does (RMS 1.4–1.9 after CSGM, against 1), but projecting it back after every Adam step does not help: rescaling each 16×16 patch to the norm of a typical Gaussian patch gains 0.13 dB but worsens LPIPS (0.373 against 0.328), and rescaling the whole $\boldsymbol{\varepsilon}$ is worse on both (21.72 dB, 0.388).

4.4 What helps: averaging reconstructions

Test set. ENS-W, the ensemble fixed before testing, improves on CSGM + BP by $+1.57$ dB and -0.032 LPIPS (Table 1; both Holm $p < 10^{-9}$). ENS-U gains $+2.13$ dB, 0.56 dB more than ENS-W, with no significant LPIPS difference (-0.005 $[-0.012, +0.002]$). Against bicubic + BP, ENS-U is 0.69 dB lower in PSNR (better on 11/100 images) but much better in LPIPS (0.247 against 0.358, -0.111 $[-0.126, -0.096]$, better on 92/100): averaging closes three quarters of CSGM's PSNR gap to bicubic (2.13 of 2.82 dB) and widens its LPIPS lead (0.111 against 0.074). Averaging raises the SNR in every band ($+1.7$ to $+2.5$ dB for ENS-U; Figure 2); above f_{LR} the members' content is worse than none, so it removes more wrong texture than true.

The averaging law. On the test set, before BP, ENS-U gains 2.33 dB $[2.18, 2.47]$ over its mean member (2.14 dB pooled over images), against 2.22 dB predicted by (7). The excess is within the CI, and the shifted members, individually worse than the restarts (by 0.19 dB, on 66/100 images), raise the mean member error: against the two restarts alone the gain is 2.19 dB. In the late-stage experiment, on six other images, 5,000-step members fall short of the prediction (1.93 dB at $K = 5$) and 500-step members exceed it (Appendix A.1). Continuing every member with IADrift@100 gives the test-set trade-off again (-0.12 dB, -0.008 LPIPS) and, with the ENS-W weights, the lowest mean LPIPS of all methods evaluated on these images (0.227; Table 3).

5 Conclusion

On this prior, CSGM is perceptually better but less accurate than bicubic, and IADrift only trades 0.12 dB of PSNR for 0.008 LPIPS. The representation error is large and adapted weights can remove it, but $\mathbf{y}$ does not constrain the null space of $\mathbf{A}$, so the weight updates change the null-space content with no information about $\mathbf{x}^\star$, mostly above f_{LR}, where the invented content is already worse than none. The failure of adaptation comes from the weights, not, as the proposal expected, from $\boldsymbol{\varepsilon}$ leaving the Gaussian shell.

Why adaptation helps IAGAN but not here. IAGAN runs our task on faces, and there IAGAN + BP beats CSGM + BP (27.14 against 26.44 dB on CelebA $\times 4$) while CSGM + BP is level with bicubic (Abu Hussein et al., 2020, Table 2). We see four differences, none of which we tested. (i) Their generators map a low-dimensional latent to aligned faces; ours maps image-sized noise and discrete labels to 1,000 classes, with no bottleneck. (ii) Part of our representation error comes from random style labels, a different kind of error from a GAN's missing modes. (iii) Our CSGM + BP is already 2.82 dB below bicubic, so the null-space content that adaptation starts from is worse than theirs. (iv) Adaptation keeps only the measurement term and drops the batch-level drifting objective (3) that defines the prior, so only the small learning rate and the step count keep the adapted generator close to p_{data}, consistent with PSNR falling from 22.63 to 19.83 dB between 100 and 1,000 steps (IAGAN also drops its adversarial term, so this alone cannot explain the difference).

DriftSR agrees. The concurrent DriftSR (Zhu et al., 2026), a supervised one-step SR network trained with drifting, meets the same problem: drifting "may encourage visually plausible textures that are inconsistent with the input". It controls them with a second, low-frequency drifting field, weighted by a map that needs the HR image, which we do not have at test time, where BP fixes only the row space. It also names the obstacle to keeping the drifting objective: drifting needs several samples per condition, while SR has one target. In our setting we propose to draw both at test time: negatives from the adapted generator and positives from the frozen one for class c, so that a drift term keeps the adapted generator close to the pretrained prior.

Challenges and next steps. The measurements cannot choose the stopping step, since the loss keeps falling while PSNR drops, so we fixed it before testing. Gradients cannot fit the discrete style labels. The ensemble came after the protocol was fixed, so only ENS-W could be added as a pre-registered test. Next: a test-set run of the late-stage ensembles. Code: `https://github.com/zivmoshe12/IADrift`.

References

Shady Abu Hussein, Tom Tirer, and Raja Giryes. Image-adaptive GAN based reconstruction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 3121–3129, 2020. doi: 10.1609/aaai.v34i04.5708.

Ashish Bora, Ajil Jalal, Eric Price, and Alexandros G. Dimakis. Compressed sensing using generative models. In *International Conference on Machine Learning (ICML)*, pages 537–546, 2017.

Mingyang Deng, He Li, Tianhong Li, Yilun Du, and Kaiming He. Generative modeling via drifting. *arXiv preprint arXiv:2602.04770*, 2026.

Wei Zhu, Kai Zhang, Yu Zheng, Zhaopeng Yang, Lei Luo, Yong Guo, and Jian Yang. DriftSR: One-step real-world image super-resolution via distribution drifting. *arXiv preprint arXiv:2610.04819*, 2026.

Appendix

A Late-stage exploration: larger ensembles

This experiment was run after the test runs, on the six development images of Section 3.3, to see how far averaging can go. It is not part of the protocol and has no significance tests. Each image has 12 CSGM members, each from its own $(\boldsymbol{\varepsilon}_r^{(0)}, \mathbf{s}_r)$, saved after 500 to 5,000 steps; the 5,000-step members are also continued with IADrift@100. Every weighted (W) average here uses the weights of ENS-W, Eq. (9), computed by the same code: $w_k \propto \exp(-(L_k - L_{\min})/\tau)$ with $\tau = \mathrm{std}(L_1, \ldots, L_K)$, where L_k is the low-resolution loss of member k in the form being averaged (for members that end with IADrift@100, the loss after those 100 steps), with one exception: the W average of "the same 5 members, each + IADrift@100" reuses the weights of ENS-W itself. Only the members and K differ from ENS-W. Table 3 lists every method evaluated on these images, and Figure 4 shows crops.

Perceptual quality. Averaging the twelve members that each end with IADrift@100, with the ENS-W weights, gives the lowest mean LPIPS of all of them (0.227). It is below twelve 5,000-step members without adaptation (0.238; lower on 4 of the 6 images) and below every other averaging variant we ran on these images, including some not reported here (all ≥ 0.238). On the four images shared with the diagnostics of Table 2 it also beats the label search and the $\boldsymbol{\varepsilon}$ projections (0.230 against 0.314, 0.373 and 0.388). Its lead is in the mean, not on every image: it is the best method on 3 of the 6 images (on the others, ENS-U, IADrift@100 or the weighted average of five IADrift@100 runs is). It was not run on the test set, so it cannot be compared with the test-set numbers of Table 1. It also costs 61.2k generator steps per image, about $6\times$ CSGM, and loses 0.19 dB PSNR against the same members without adaptation. Bicubic + BP keeps the highest PSNR on all six images.

Run length. Longer members mainly buy LPIPS: twelve members reach about 25.1 dB at every run length from 500 to 5,000 steps, while their uniform LPIPS falls from 0.315 to 0.246 (Figure 3b).

Table 3: Late-stage exploration: 6 development images, all after BP, means over images; no significance tests. Steps: Adam steps through the generator per image. U / W: uniform / weighted average (weights of ENS-W). Five-member rows marked * are means over 20 random 5-member subsets of the 12. Bold: best.

Method	Steps	PSNR ↑ (U / W)	LPIPS ↓ (U / W)
Bicubic	–	**25.73**	0.418
CSGM	10k	22.45	0.303
IADrift@100	10.1k	22.14	0.304
ENS-U / ENS-W (Section 3.3)	25k	24.81 / 24.04	0.302 / 0.273
The same 5 members, each + IADrift@100	25.5k	24.71 / 23.82	0.293 / 0.273
12×500 steps	6k	25.05 / 24.92	0.315 / 0.307
$12 \times 1{,}000$ steps	12k	25.10 / 25.01	0.299 / 0.286
$12 \times 2{,}000$ steps	24k	25.12 / 25.05	0.278 / 0.265
$5 \times 5{,}000$ steps*	25k	24.74 / 24.45	0.261 / 0.249
$12 \times 5{,}000$ steps	60k	25.10 / 24.96	0.246 / 0.238
$5 \times (5{,}000 + \text{IADrift@100})$*	25.5k	24.59 / 24.15	0.249 / 0.244
$12 \times (5{,}000 + \text{IADrift@100})$	61.2k	24.98 / 24.77	0.238 / **0.227**

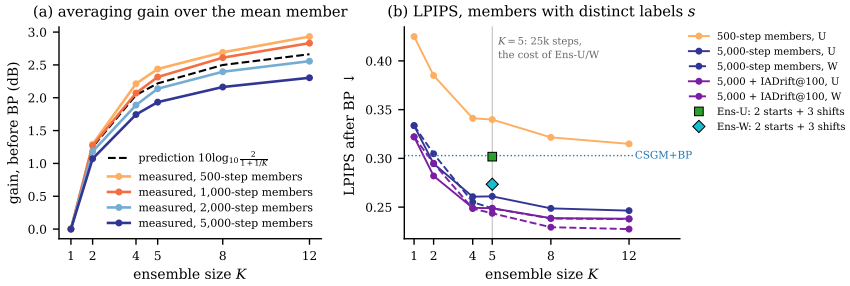


Figure 3: Late-stage exploration, 6 development images; each point averages up to 20 random K-subsets of 12 members. *(a)* Gain of a uniform average over the mean member, before BP, against (7). *(b)* LPIPS after BP; the markers at $K = 5$ are ENS-U and ENS-W on the same images.

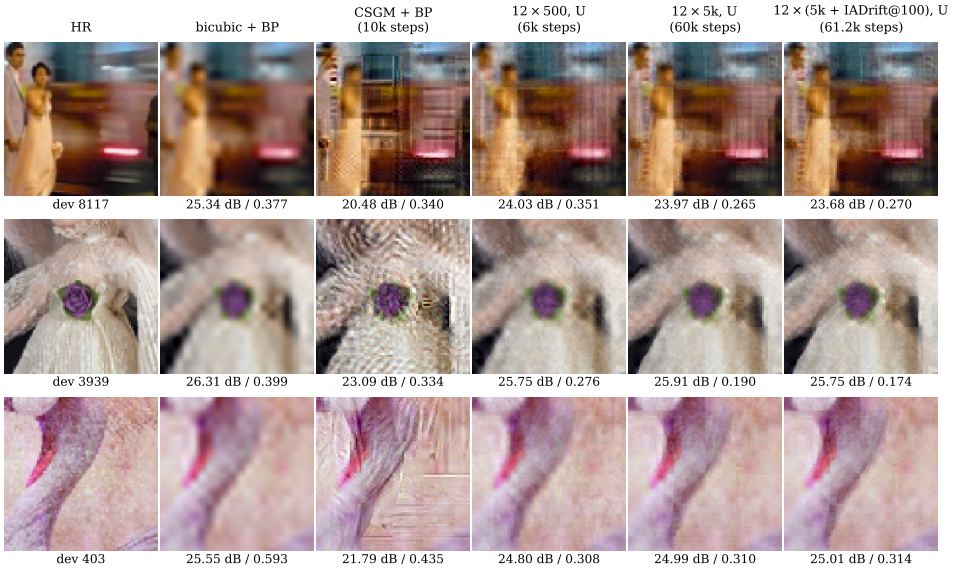


Figure 4: Late-stage exploration, the first three of the six development images (not chosen by result), after BP; zoomed 80×80 centre crops. Below each crop: PSNR / LPIPS of the full image. The last three columns are uniform averages of the 12 members, at 500 and 5,000 CSGM steps and after a further 100 IADrift steps each (the weighted averages are not drawn, since their member images were not exported).

A.1 Why the averaging gain departs from $2/(1 + 1/K)$

The prediction (7) assumes that the truth and the members are independent posterior samples. The measured gains of Figure 3a fall short of it for long runs and exceed it for short ones. On the test set ENS-U gains 2.33 dB, above it, but within the CI and with shifted members that are individually worse, which raises the mean member error (Section 4.4). Three models describe the departure; none of them was tested beyond the fits below.

Correlated members. If $\mathbb{E}[\mathbf{u}_k^\top \mathbf{u}_l] = \rho \operatorname{tr}\Sigma$ for $k \neq l$,

$$\mathbb{E}\left\| \tfrac{1}{K}\textstyle\sum_k \mathbf{x}_k - \mathbf{x}^\star \right\|^2 = \left(1 + \tfrac{1}{K} + \tfrac{(K-1)\rho}{K}\right)\operatorname{tr}\Sigma, \qquad \text{gain} = 10\log_{10}\frac{2}{1+1/K+(K-1)\rho/K}. \quad (10)$$

Fitted per K, the measured gains give $\rho = 0.10$–0.13 for 5,000-step members (1.07, 1.74, 1.93, 2.16 and 2.30 dB at $K = 2, 4, 5, 8, 12$, against 1.25, 2.04, 2.22, 2.50 and 2.66 dB predicted) and

$\rho = 0.03$–0.06 for 2,000-step members. The fit cannot tell a true correlation from a shared bias, such as a prior whose mean is not the posterior mean: both add an error term that averaging does not remove.

Averaging over labels as a mixture. The labels are a discrete latent of the prior, so the posterior splits over them. Given $(\boldsymbol{\varepsilon}, \mathbf{s})$ the generator is deterministic, $p(\mathbf{x} \mid \boldsymbol{\varepsilon}, \mathbf{s}) = \delta\big(\mathbf{x} - G_{\boldsymbol{\theta}_0}(\boldsymbol{\varepsilon}, \mathbf{s}; c)\big)$, and therefore

$$\begin{gathered} p(\mathbf{x} \mid \mathbf{y}) = \sum_{\mathbf{s} \in \mathcal{S}} p(\mathbf{s} \mid \mathbf{y})\, p(\mathbf{x} \mid \mathbf{y}, \mathbf{s}), \qquad \boldsymbol{\mu} = \sum_{\mathbf{s} \in \mathcal{S}} p(\mathbf{s} \mid \mathbf{y})\, \mathbf{m}(\mathbf{s}), \\ \mathbf{m}(\mathbf{s}) = \mathbb{E}[\mathbf{x} \mid \mathbf{y}, \mathbf{s}] = \mathbb{E}_{\boldsymbol{\varepsilon} \sim p(\boldsymbol{\varepsilon} \mid \mathbf{y}, \mathbf{s})}\big[G_{\boldsymbol{\theta}_0}(\boldsymbol{\varepsilon}, \mathbf{s}; c)\big], \end{gathered} \tag{11}$$

with $|\mathcal{S}| = 64^{32}$. The weights are $p(\mathbf{s} \mid \mathbf{y}) \propto p(\mathbf{s})\, p(\mathbf{y} \mid \mathbf{s})$, not the prior, because $\mathbf{y}$ carries information about the labels: the label search of Table 2 lowers the final low-resolution loss from 50.5 to 30.0. A uniform average of members with $\mathbf{s}_k \sim \mathrm{Unif}(\mathcal{S})$ is a Monte Carlo estimate of $\boldsymbol{\mu}$ with the prior as weights, and the weights of (9) act like $p(\mathbf{y} \mid \mathbf{s}_k)$ in self-normalized importance sampling, but only loosely: with $\mathbf{e} = \mathbf{0}$, $p(\mathbf{y} \mid \mathbf{s})$ has no closed form, and CSGM fits $\boldsymbol{\varepsilon}$ without its prior, so a member is a point of the fit set reached from a random start, not $\mathbf{m}(\mathbf{s}_k)$.

Still, the mixture changes the prediction. Suppose each member is exactly $\mathbf{m}(\mathbf{s}_k)$ with $\mathbf{s}_k \sim p(\mathbf{s} \mid \mathbf{y})$ i.i.d. Let $W = \mathbb{E}_{\mathbf{s}} \operatorname{tr} \mathrm{Cov}(\mathbf{x} \mid \mathbf{y}, \mathbf{s})$ be the variance within labels and $B = \operatorname{tr} \mathrm{Cov}_{\mathbf{s}}\, \mathbf{m}(\mathbf{s})$ the variance between them, so that $W + B = \operatorname{tr} \Sigma$ by the law of total variance, and let $\beta = B / \operatorname{tr} \Sigma$. Then

$$\begin{gathered} \mathbb{E}\|\mathbf{x}_k - \mathbf{x}^\star\|^2 = (1 + \beta) \operatorname{tr} \Sigma, \qquad \mathbb{E}\Big\| \tfrac{1}{K} \textstyle\sum_k \mathbf{x}_k - \mathbf{x}^\star \Big\|^2 = \big(1 + \tfrac{\beta}{K}\big) \operatorname{tr} \Sigma, \\ \text{gain} = 10 \log_{10} \frac{1 + \beta}{1 + \beta/K}. \end{gathered} \tag{12}$$

For $\beta = 1$ this is (7). For $\beta < 1$ the variance within labels is in the error of every member and of the average alike, so the gain stays below (7) and saturates at $10 \log_{10}(1 + \beta) < 3$ dB. Algebraically, (12) is (10) with $\rho = (1 - \beta)/(1 + \beta)$, so the two models fit the measured gains equally well but read the shortfall differently. For 5,000-step members the shortfall gives $\beta = 0.78$–0.82: if each member were the mean given its labels, about a fifth of the posterior variance would lie within labels, where no label ensemble can reach it. Our members also differ in their start $\boldsymbol{\varepsilon}^{(0)}$, which covers part of the spread within labels; (12) is the idealized case in which it does not.

Optimization error. Short-run members *exceed* the prediction (by 0.04–0.27 dB at 500 steps; 2.93 against 2.66 dB at $K = 12$), which neither model above allows for $\rho \geq 0$ or $\beta \leq 1$. A post-hoc model is consistent with this: write $\mathbf{x}_k = \boldsymbol{\mu} + \mathbf{u}_k + \mathbf{v}_k$, where $\mathbf{v}_k$ is an independent optimization error with $\mathbb{E}\|\mathbf{v}_k\|^2 = a \operatorname{tr} \Sigma$. Then

$$\begin{gathered} \mathbb{E}\|\mathbf{x}_k - \mathbf{x}^\star\|^2 = (2 + a) \operatorname{tr} \Sigma, \qquad \mathbb{E}\Big\| \tfrac{1}{K} \textstyle\sum_k \mathbf{x}_k - \mathbf{x}^\star \Big\|^2 = \Big(1 + \tfrac{1 + a}{K}\Big) \operatorname{tr} \Sigma, \\ \text{gain} = 10 \log_{10} \frac{2 + a}{1 + (1 + a)/K} > 10 \log_{10} \frac{2}{1 + 1/K}, \end{gathered} \tag{13}$$

because averaging removes $\mathbf{v}_k$ as well. We did not estimate a, so this is not a test. The sign of the difference flips between 1,000 and 2,000 steps: at 1,000 steps the gain exceeds the prediction at every K (by 0.02–0.17 dB), and at 2,000 steps it falls short at every K (by 0.07–0.15 dB). Read through (13) and (10), the optimization error dominates early and the correlation or shared bias late.